

Trilha 6 - Inglês Técnico para Engenharia de Software em Nível Sênior

Plano de estudos técnico, progressivo e aplicado para atuação internacional em engenharia de software

Objetivo central: atingir B2+ funcional para trabalhar em times internacionais, comunicar decisões técnicas, participar de entrevistas, reuniões, incidentes, code reviews e negociações.

Conteúdo organizado como trilha de evolução profissional: fundamentos, aplicação prática, documentação, arquitetura, produção, liderança técnica e entrevistas internacionais.

Fonte: conteúdo textual anexado pelo usuário. Formatação: PDF paginado com sumário, hierarquia de títulos, tabelas e blocos de exemplo.

Versão formatada - 07/06/2026

Sumário

Lista das principais seções com paginação gerada na versão padronizada.

1. Análise inicial da trilha	4
1. Reading técnico	4
2. Writing técnico	4
3. Speaking técnico	5
4. Listening avançado	5
5. Comunicação de liderança técnica	5
2. Explicação técnica de cada tópico	5
2.1. Leitura técnica	5
2.2. Escrita técnica	7
2.3. Speaking técnico	10
2.4. Vocabulário de liderança técnica	13
2.5. Listening avançado	15
2.6. Entrevistas internacionais	16
3. Organização como plano de estudos	18
Módulo 1 - Fundamentos do Inglês Técnico	18
Módulo 2 - Aplicação prática em rotina de desenvolvimento	20
Módulo 3 - Aprofundamento em documentação e comunicação técnica	21
Módulo 4 - Arquitetura, boas práticas e liderança técnica	23
Módulo 5 - Produção, performance, segurança e escalabilidade	25
Módulo 6 - Domínio sênior e entrevistas internacionais	27
Projeto principal em inglês.	27
4. Critérios de domínio técnico por grande área	29
5. Aplicação prática em projetos reais	30
6. Profundidade sênior por tópico	34
7. Plano de execução sugerido	36
Checkpoint 1 - Semana 4	37

Checkpoint 2 - Semana 8 37

Checkpoint 3 - Semana 12 38

Checkpoint 4 - Semana 16 38

Checkpoint 5 - Semana 20 38

Checkpoint 6 - Semana 24 38

8. Projetos recomendados para portfólio 38

Projeto 1 - API documentada em inglês 38

Projeto 2 - ADR collection 39

Projeto 3 - Incident simulation 39

Projeto 4 - System design presentation 39

Projeto 5 - International interview package 39

9. Checklist de domínio técnico 40

10. Indicadores de prontidão para nível avançado/sênior 41

1. Análise inicial da trilha

Tema central

A trilha é focada em Inglês Técnico aplicado à atuação profissional em tecnologia, especialmente para desenvolvedores, arquitetos, engenheiros backend/full stack, DevOps, cloud engineers, tech leads e profissionais que desejam trabalhar em times internacionais.

O objetivo não é apenas "aprender inglês", mas desenvolver fluência funcional em contextos técnicos reais, como:

- reuniões de engenharia;
- entrevistas internacionais;
- code reviews;
- incidentes em produção;
- documentação técnica;
- decisões arquiteturais;
- system design;
- negociação de escopo, prazos e salário;
- colaboração com times distribuídos.

Objetivo principal da trilha

Chegar a um nível B2+ funcional, com capacidade de trabalhar em inglês sem depender de tradução mental constante.

Isso significa conseguir:

- ler documentação oficial e aplicar em projeto real;
- escrever documentação técnica clara;
- explicar bugs, features e decisões arquiteturais;
- participar de reuniões técnicas;
- conduzir discussões de code review;
- defender trade-offs;
- participar de entrevistas internacionais;
- entender sotaques variados;
- responder de forma objetiva, técnica e profissional.

Competências técnicas a desenvolver

Esta trilha desenvolve cinco grandes competências:

1. Reading técnico

Capacidade de ler documentação, RFCs, issues, pull requests, whitepapers, ADRs e artigos técnicos sem depender de tradução.

2. Writing técnico

Capacidade de escrever mensagens, documentos e propostas técnicas com clareza, precisão e tom profissional.

3. Speaking técnico

Capacidade de falar sobre problemas, soluções, arquitetura, estimativas, incidentes e decisões técnicas em reuniões reais.

4. Listening avançado

Capacidade de entender reuniões, entrevistas, podcasts, talks e discussões técnicas com sotaques diversos.

5. Comunicação de liderança técnica

Capacidade de influenciar, justificar decisões, recusar abordagens ruins, negociar escopo e conduzir discussões de engenharia.

Classificação por nível

Nível O que representa nesta trilha

Básico Entende termos técnicos isolados, lê pequenos trechos de documentação e monta frases simples.

Intermediário Consegue ler documentação com apoio, escrever commits/PRs simples e explicar tarefas técnicas conhecidas.

Avançado Consegue discutir bugs, features, arquitetura e incidentes com boa autonomia.

Sênior Consegue liderar discussões técnicas, justificar trade-offs, conduzir entrevistas, escrever documentos estratégicos e atuar em times internacionais.

2. Explicação técnica de cada tópico

2.1. Leitura técnica

O que é

Leitura técnica é a capacidade de consumir materiais reais de engenharia de software em inglês, como:

- documentação oficial;
- issues no GitHub;
- pull requests;
- RFCs;
- ADRs;
- postmortems;
- whitepapers;
- documentação AWS;
- documentação Spring;
- documentação Kubernetes;
- artigos técnicos;
- papers simplificados.

- Não é leitura escolar. É leitura aplicada a decisões, implementação e resolução de problemas.

Para que serve

Serve para permitir que o profissional consiga:

- entender como uma tecnologia funciona;
- aprender recursos novos diretamente da fonte;
- resolver bugs lendo documentação oficial;
- avaliar decisões arquiteturais;
- entender limitações de ferramentas;
- acompanhar discussões técnicas em comunidades globais;
- investigar incidentes e problemas de produção.

Por que é importante

Grande parte do conhecimento técnico de alto nível está em inglês. Documentação oficial, especificações, RFCs, artigos de engenharia, discussões no GitHub e materiais de cloud são majoritariamente produzidos em inglês.

Um profissional sênior não pode depender apenas de vídeos ou conteúdos traduzidos. Ele precisa acessar a fonte primária.

Exemplo prático: ao trabalhar com Spring Security, Kafka, Kubernetes ou AWS, muitas respostas específicas não estarão em cursos introdutórios. Estarão em documentação, issues, changelogs e discussões técnicas.

Onde é usado em projetos reais

Em projetos reais, leitura técnica aparece quando você precisa:

- configurar autenticação OAuth2/OIDC no Spring Security;
- entender comportamento de retry no Kafka;
- interpretar erro de deploy no Kubernetes;
- avaliar limites de um serviço AWS;
- revisar um ADR escrito por outro arquiteto;
- investigar uma issue aberta em biblioteca open source;
- entender breaking changes em uma nova versão de framework;
- analisar postmortem de incidente para evitar falhas semelhantes.

Quais problemas resolve

A leitura técnica resolve problemas como:

- dependência excessiva de tutoriais;
- cópia de código sem entendimento;
- dificuldade para investigar erros complexos;
- lentidão para aprender tecnologias novas;
- dificuldade em justificar decisões técnicas;
- baixa autonomia em ambientes internacionais.

Erros comuns

Erros frequentes:

- traduzir palavra por palavra;
- tentar entender 100% do texto antes de aplicar;
- ignorar termos técnicos recorrentes;

- consumir apenas conteúdo didático, nunca documentação real;
- pular seções como limitations, caveats, migration guide e security considerations;
- não anotar expressões úteis;
- não transformar leitura em prática.

Conhecimentos prévios necessários

Antes de avançar bem em leitura técnica, o estudante precisa ter:

- vocabulário básico de tecnologia;
- noções de gramática essencial;
- familiaridade com termos como request, response, dependency, failure, timeout, retry, deployment, throughput, latency, consistency;
- conhecimento técnico mínimo da área estudada.
- Por exemplo: para ler documentação de Kubernetes, é necessário entender pelo menos o básico de containers, pods, services, deployments e networking.

Profundidade esperada para nível sênior

Um profissional sênior deve conseguir:

- ler documentação oficial e aplicar sem tradução;
- identificar trechos críticos;
- diferenciar recomendação, restrição e aviso;
- entender limitações operacionais;
- extrair decisões arquiteturais de textos técnicos;
- comparar abordagens entre ferramentas;
- ler postmortems e aplicar lições aprendidas;
- usar documentação como base para defender decisões técnicas.
- Critério sênior: ler, interpretar, aplicar e explicar a documentação para outras pessoas.

2.2. Escrita técnica

O que é

Escrita técnica é a habilidade de produzir comunicação escrita clara, objetiva e profissional sobre software.

Inclui:

- commit messages;
- pull request descriptions;
- README;
- ADR;
- RFC;
- bug report;
- incident report;
- technical proposal;
- API documentation;
- architecture documentation;
- release notes.

Para que serve

Serve para registrar decisões, orientar times, documentar mudanças, reduzir ruído de comunicação e facilitar manutenção futura.

Em times internacionais, boa escrita técnica é uma das habilidades mais importantes porque boa parte da comunicação acontece de forma assíncrona.

Por que é importante

Código sem contexto envelhece mal. Sistemas reais têm decisões, restrições, incidentes, trade-offs e histórico. A escrita técnica preserva esse conhecimento.

Um profissional sênior precisa escrever de forma que outro engenheiro consiga entender:

- o que foi feito;
- por que foi feito;
- quais alternativas foram consideradas;
- quais riscos existem;
- como validar;
- como fazer rollback;
- quais impactos existem em produção.

Onde é usado em projetos reais

Commit message

Usado para explicar uma alteração específica.

Exemplo:

```
fix: handle null fiscal document when mapping tax receipt request
```

Pull request description

Usado para explicar contexto, motivação, testes e impacto da mudança.

Exemplo:

```
This change introduces null-safe mapping for fiscal document fields.
```

```
It prevents NullPointerException when the upstream service sends incomplete payloads.
```

- ADR
- Usado para registrar decisão arquitetural.

Exemplo: escolher Kafka em vez de chamada síncrona HTTP para desacoplar processamento de pedidos.

```
RFC
```

Usado para propor uma mudança relevante antes da implementação.

Exemplo: criação de um novo fluxo de autenticação com OAuth2 e refresh token.

```
Incident report
```

Usado para registrar causa raiz, impacto, mitigação e ações preventivas.

Quais problemas resolve

A escrita técnica reduz:

- retrabalho;
- dúvidas repetidas;
- decisões sem histórico;

- PRs difíceis de revisar;
- deploys arriscados;
- desalinhamento entre times;
- dependência de conhecimento verbal;
- incidentes recorrentes.

Erros comuns

Erros comuns:

- escrever commits genéricos como fix, adjust, changes;
- abrir PR sem contexto;
- documentar apenas "o que" foi feito, sem explicar "por quê";
- escrever ADR depois que a decisão já virou problema;
- usar texto longo, confuso e sem estrutura;
- exagerar no vocabulário rebuscado;
- omitir riscos, limitações e plano de rollback;
- não documentar breaking changes.

Conhecimentos prévios necessários

Para escrever bem, o estudante precisa dominar:

- gramática funcional;
- vocabulário técnico;
- estrutura de frases objetivas;
- conectores de causa, consequência e contraste;
- noções de arquitetura e ciclo de desenvolvimento;
- entendimento do público-alvo do documento.
- Modelos de frases essenciais
- Para introduzir mudança

This change introduces...

Uso: explicar o que uma alteração adiciona.

Exemplo:

This change introduces a retry mechanism for failed payment events.

Para explicar trade-off

The main trade-off is...

Exemplo:

The main trade-off is increased infrastructure complexity in exchange for better fault tolerance.

Para explicar benefício

This approach reduces...

Exemplo:

This approach reduces coupling between the order service and the billing service.

Para explicar causa raiz

The root cause was...

Exemplo:

The root cause was an invalid assumption about the payload structure received from the upstream provider.

Para propor solução

The proposed solution is...

Exemplo:

The proposed solution is to validate the payload before mapping it to the internal domain model.

Para compatibilidade

This is backward compatible because...

Exemplo:

This is backward compatible because the existing API contract remains unchanged.

Para rollout

The rollout plan is...

Exemplo:

The rollout plan is to enable the feature flag for internal users first, monitor error rates, and then gradually increase traffic.

Profundidade esperada para nível sênior

Um sênior deve conseguir escrever:

- PRs claros e revisáveis;
- ADRs objetivos;
- RFCs com trade-offs;
- relatórios de incidente sem culpabilização;
- documentação de arquitetura;
- propostas técnicas para stakeholders;
- release notes com impacto real;
- documentação de API compreensível para outros times.
- Critério sênior: escrever documentos técnicos que ajudem o time a decidir, implementar, operar e manter software.

2.3. Speaking técnico

O que é

Speaking técnico é a capacidade de falar em inglês sobre trabalho real de engenharia.

Inclui participar de:

- daily standup;
- sprint planning;
- sprint review;
- retrospective;
- architecture review;
- incident call;
- pair programming;
- code review discussion;

- technical interview;
- system design interview;
- salary negotiation.

Para que serve

Serve para colaborar em tempo real com times internacionais.

O profissional precisa explicar o que está fazendo, pedir ajuda, defender decisões, alinhar escopo, conduzir discussões e responder perguntas técnicas sem travar.

Por que é importante

Em ambientes internacionais, saber programar não basta. O profissional precisa transformar raciocínio técnico em comunicação clara.

Um desenvolvedor avançado pode escrever bom código. Um sênior precisa explicar por que aquele código, aquela arquitetura ou aquela abordagem fazem sentido.

Onde é usado em projetos reais

Daily standup

Exemplo:

Yesterday I worked on the order validation flow.

Today I'll finish the integration tests and start reviewing the Kafka retry strategy.

I'm blocked by an unclear requirement from the billing team.

Code review discussion

Exemplo:

I'd suggest extracting this validation into a separate component because this service is starting to handle too many responsibilities.

Architecture review

Exemplo:

The main concern with this approach is operational complexity. It improves scalability, but it also requires better observability and a clear rollback strategy.

Incident call

Exemplo:

The error rate started increasing after the last deployment. We are rolling back the service and checking whether the issue is related to the new payload validation logic.

Quais problemas resolve

Speaking técnico resolve:

- dificuldade de participar de reuniões;
- baixa confiança em entrevistas internacionais;
- comunicação lenta com times globais;
- incapacidade de explicar decisões;
- dependência de mensagens escritas para tudo;
- dificuldade de negociar prazo, escopo e prioridade.

Erros comuns

Erros frequentes:

- tentar traduzir mentalmente do português;
- decorar frases sem entender estrutura;
- falar de forma longa e confusa;
- evitar reuniões por insegurança;
- não praticar cenários reais;
- focar apenas em pronúncia perfeita;
- não saber pedir clarificação;
- não saber discordar com educação.

Conhecimentos prévios necessários

Para falar tecnicamente, o estudante precisa:

- vocabulário técnico ativo;
- estruturas básicas de presente, passado e futuro;
- conectores como because, although, however, therefore, in order to;
- prática de frases curtas;
- familiaridade com o próprio trabalho técnico;
- capacidade de explicar conceitos em português antes de explicar em inglês.

Temas obrigatórios de prática

Explicar um bug

The bug happens when the request payload does not include the fiscal document field. The mapper assumes that the field is always present, which causes a null pointer exception.

Explicar uma feature

This feature allows users to upload transaction files and automatically classify expenses based on predefined categories.

Explicar uma decisão arquitetural

We chose asynchronous communication because the payment flow should not block the order creation process.

Defender estimativa

The estimate includes implementation, tests, integration validation, documentation, and deployment monitoring.

Recusar escopo inviável

I don't think we should include this in the current sprint because it changes the API contract and increases the risk of regression.

Explicar trade-off

The trade-off is that we gain scalability, but we introduce more operational complexity.

Conduzir postmortem

The goal of this postmortem is to understand what happened, identify contributing factors, and define preventive actions.

Apresentar portfólio

This project is a financial dashboard built with Next.js on the frontend and Spring Boot on the backend. The main goal was to provide transaction tracking, budget management, and insights based on user spending patterns.

Profundidade esperada para nível sênior

Um profissional sênior deve conseguir:

- falar de forma espontânea;
- conduzir discussões técnicas;
- explicar decisões sob pressão;
- lidar com perguntas inesperadas;
- discordar profissionalmente;
- negociar escopo;
- explicar impacto em produção;
- apresentar arquitetura;
- participar de entrevistas internacionais.
- Critério sênior: falar sem montar a frase em português primeiro.

2.4. Vocabulário de liderança técnica

O que é

É o conjunto de expressões usadas por profissionais experientes para liderar discussões, apontar riscos, negociar decisões e alinhar times.

Não é apenas vocabulário técnico. É linguagem de influência técnica.

Para que serve

Serve para:

- questionar decisões ruins;
- defender qualidade;
- explicar riscos;
- propor alternativas;
- alinhar estratégia;
- conduzir revisões;
- evitar problemas em produção;
- demonstrar maturidade profissional.

Expressões essenciais e aplicação

"This is a blocking issue."

Usada quando algo impede o avanço do trabalho.

Exemplo:

This is a blocking issue because we cannot deploy the service until the authentication flow is fixed.

"I'd push back on this approach."

Usada para discordar de forma profissional.

Exemplo:

I'd push back on this approach because it increases coupling between services and makes future changes harder.

"The trade-off here is..."

Usada para apresentar custo e benefício.

Exemplo:

The trade-off here is faster delivery versus long-term maintainability.

"From an operational perspective..."

Usada para avaliar impacto em produção.

Exemplo:

From an operational perspective, this solution requires better monitoring and a clear rollback process.

"This might introduce coupling."

Usada para alertar sobre dependência excessiva.

Exemplo:

This might introduce coupling between the checkout service and the payment provider.

"We should validate this assumption."

Usada para evitar decisões baseadas em suposições.

Exemplo:

We should validate this assumption before changing the database schema.

"The risk is acceptable if..."

Usada para condicionar aceitação de risco.

Exemplo:

The risk is acceptable if we release it behind a feature flag and monitor the error rate.

"Let's align on the rollout strategy."

Usada para planejar implantação.

Exemplo:

Let's align on the rollout strategy before enabling this feature for all users.

"We need a rollback plan."

Usada para exigir plano de reversão.

Exemplo:

We need a rollback plan because this change affects the payment flow.

"This is not production-ready yet."

Usada para indicar que algo ainda não está seguro para produção.

Exemplo:

This is not production-ready yet because we still don't have enough automated tests and monitoring.

Erros comuns

- usar linguagem agressiva ao discordar;
- dizer apenas "I don't like it" sem justificativa técnica;
- não explicar impacto;
- criticar sem propor alternativa;
- não diferenciar risco aceitável de risco crítico;
- aceitar decisões ruins por insegurança no inglês;
- não usar vocabulário de produção, como rollback, monitoring, failure rate, latency, throughput, coupling.

Profundidade esperada para nível sênior

Um sênior deve usar inglês para:

- liderar discussões técnicas;
- influenciar decisões;
- proteger qualidade;
- negociar prazo e escopo;
- orientar o time;
- comunicar riscos;
- justificar escolhas arquiteturais;
- alinhar engenharia com produto e negócio.
- Critério sênior: conseguir ser tecnicamente firme sem ser rude.

2.5. Listening avançado

O que é

Listening avançado é a capacidade de entender inglês falado em contextos reais, com diferentes sotaques, velocidades, vocabulários e níveis de clareza.

A prática deve incluir:

- sotaque americano;
- britânico;
- indiano;
- australiano;
- inglês de não nativos;
- reuniões gravadas;
- podcasts técnicos;
- talks de arquitetura;
- entrevistas simuladas.

Para que serve

Serve para acompanhar reuniões, entrevistas e discussões técnicas sem depender de legenda.

Em times internacionais, o desafio não é apenas entender inglês "perfeito". É entender pessoas reais falando com sotaques, ruídos, pausas, gírias e termos técnicos.

Por que é importante

Você pode ter boa leitura e escrita, mas travar em reunião se não entender perguntas, objeções ou decisões.

Listening avançado é essencial para:

- responder no fluxo;
- não perder decisões importantes;
- participar de incident calls;
- entender entrevistas;
- colaborar com pessoas de diferentes países;
- evitar mal-entendidos técnicos.

Onde é usado em projetos reais

- reunião de refinamento com product manager;

- call de incidente com SRE;
- system design interview;
- pair programming com engenheiro estrangeiro;
- discussão de arquitetura com arquiteto global;
- retrospectiva com time distribuído;
- apresentação de roadmap técnico.

Erros comuns

- treinar apenas com vídeos legendados;
- ouvir apenas sotaque americano claro;
- não praticar reuniões reais;
- pausar a cada frase;
- querer entender 100% das palavras;
- ignorar contexto;
- não anotar expressões recorrentes;
- não praticar resposta oral depois de ouvir.

Conhecimentos prévios necessários

- vocabulário técnico intermediário;
- familiaridade com temas de engenharia;
- exposição regular ao inglês falado;
- conhecimento de expressões de reunião;
- tolerância a ambiguidade.

Profundidade esperada para nível sênior

Um profissional sênior deve conseguir:

- entender reunião técnica sem legenda;
- identificar decisões e próximos passos;
- perceber riscos levantados;
- responder perguntas no fluxo;
- pedir clarificação de forma natural;
- entender diferentes sotaques;
- conduzir parte da reunião quando necessário.
- Critério sênior: entender, processar e responder sem interromper o fluxo da reunião.

2.6. Entrevistas internacionais

O que é

Preparação para processos seletivos em inglês, incluindo:

- behavioral interview;
- system design;
- live coding;
- project deep dive;
- architecture trade-offs;
- conflict resolution;
- incident response;

- leadership examples;
- salary negotiation.

Para que serve

Serve para demonstrar domínio técnico, comunicação, maturidade e capacidade de atuação em ambientes internacionais.

Em entrevistas internacionais, não basta acertar tecnicamente. É necessário explicar o raciocínio com clareza.

Principais formatos

Behavioral interview

Avalia comportamento, colaboração, conflitos, liderança e maturidade.

Exemplo de pergunta:

Tell me about a time you had to deal with a production incident.

System design interview

Avalia arquitetura, escalabilidade, trade-offs, dados, APIs, cache, filas, observabilidade e segurança.

Exemplo:

Design a scalable order management system.

Live coding

Avalia lógica, estruturas de dados, clareza, comunicação e qualidade do código.

Project deep dive

Avalia profundidade real sobre projetos anteriores.

Exemplo:

Can you walk me through the architecture of one of your most relevant projects?

Salary negotiation

Avalia posicionamento profissional e clareza de expectativa.

Exemplo:

Based on the role scope and my experience, I'm targeting a compensation range aligned with senior backend positions in international teams.

Entregáveis obrigatórios

Ao final da preparação, o estudante deve ter:

- pitch pessoal em inglês;
- explicação de um projeto principal em inglês;
- 10 respostas STAR;
- 5 system designs apresentados oralmente;
- 20 mock interviews.

Erros comuns

- decorar respostas prontas;
- não saber explicar projetos próprios;
- falar apenas "I worked with Java and Spring" sem profundidade;
- não demonstrar impacto;
- não explicar trade-offs;

- não praticar perguntas comportamentais;
- não saber negociar salário;
- falar muito sobre tecnologia e pouco sobre resultado.

Profundidade esperada para nível sênior

Um sênior deve conseguir:

- contar experiências com contexto, ação e resultado;
 - explicar decisões arquiteturais;
 - reconhecer falhas e aprendizados;
 - defender trade-offs;
 - desenhar sistemas escaláveis;
 - discutir segurança, observabilidade, performance e custos;
 - negociar com clareza e profissionalismo.
- Critério sênior: ser avaliado como alguém capaz de atuar com autonomia técnica e comunicação madura em um time internacional.

3. Organização como plano de estudos

Módulo 1 - Fundamentos do Inglês Técnico

Objetivo do módulo

Construir a base mínima para ler, escrever e falar sobre desenvolvimento de software em inglês.

Conteúdos a estudar

Vocabulário essencial de software.

Estruturas gramaticais funcionais.

Frases comuns em reuniões.

Termos de Git, API, banco de dados, cloud, testes e deploy.

Leitura curta de documentação.

Escrita de frases técnicas simples.

Ordem recomendada

Vocabulário técnico básico.

Verbos comuns em tecnologia: create, update, delete, fetch, handle, validate, deploy, monitor, rollback.

Frases de rotina.

Leitura de documentação simples.

Escrita de commits e comentários técnicos.

Speaking com frases curtas.

Conceitos obrigatórios

Present simple.

Past simple.

Future with will/going to.

Modal verbs: should, must, can, could, might.

Cause and effect: because, therefore, due to.

Contrast: however, although, but.

Sequencing: first, then, after that, finally.

Práticas recomendadas

Ler 15 minutos de documentação por dia.

Criar glossário técnico pessoal.

Escrever 3 commits fictícios por dia.

Explicar uma tarefa simples em voz alta.

Gravar áudio de 1 minuto explicando o que estudou.

Exercícios técnicos

Traduzir mentalmente termos técnicos para conceitos, não para palavras.

Escrever 10 frases usando This change introduces....

Explicar uma API REST simples em inglês.

Ler uma página curta da documentação do Spring.

Resumir um endpoint em 5 linhas.

Mini projeto

Criar um README em inglês para uma API simples de tarefas.

Deve conter:

- project overview;
- features;
- technologies;
- how to run;
- API endpoints;
- testing;
- future improvements.

Critérios de conclusão

O módulo está concluído quando o estudante consegue:

- ler documentação simples;
- escrever commits claros;
- explicar uma tarefa básica;
- entender perguntas simples em reunião;
- escrever um README técnico curto.

Nível esperado ao finalizar

Básico técnico sólido, entre A2+ e B1 funcional em contexto de software.

Módulo 2 - Aplicação prática em rotina de desenvolvimento

Objetivo do módulo

Usar inglês no fluxo real de trabalho: commits, PRs, code reviews, daily, planning e bugs.

Conteúdos a estudar

Commit messages.

Pull request descriptions.

Bug reports.

Daily standup.

Code review comments.

Sprint planning.

Explicação de bugs e features.

Ordem recomendada

Commits.

PR descriptions.

Bug reports.

Code review comments.

Daily updates.

Explicação oral de bugs.

Simulações de planning.

Conceitos obrigatórios

Descrever mudança.

Explicar motivo.

Explicar impacto.

Informar risco.

Pedir revisão.

Apontar bloqueio.

Sugerir melhoria.

Práticas recomendadas

Escrever todo commit de estudo em inglês.

Criar descrição completa para cada PR fictício.

Simular daily 3 vezes por semana.

Fazer code review em inglês de códigos próprios.

Documentar bugs reais ou simulados.

Exercícios técnicos

Escrever 20 commits técnicos.

Criar 5 descrições de PR.

Criar 5 bug reports.

Explicar oralmente 5 bugs.

Gravar uma daily de 2 minutos.

Escrever comentários de code review para código Java/Spring.

Mini projeto

Criar uma API Spring Boot simples e documentar todo o fluxo em inglês:

- commits;
- pull request;
- README;
- bug report;
- release notes;
- code review comments.

Critérios de conclusão

O módulo está concluído quando o estudante consegue:

- explicar o que fez ontem, o que fará hoje e bloqueios;
- escrever PR com contexto;
- reportar bug com passos de reprodução;
- comentar código em inglês;
- defender uma pequena decisão técnica.

Nível esperado ao finalizar

Intermediário inicial, próximo de B1/B1+ técnico.

Módulo 3 - Aprofundamento em documentação e comunicação técnica

Objetivo do módulo

Desenvolver capacidade de ler e escrever documentação mais densa, incluindo ADRs, RFCs, documentação de API e propostas técnicas.

Conteúdos a estudar

- ADR.
- RFC.
- API documentation.
- Architecture documentation.
- Technical proposal.
- Release notes.
- Postmortem básico.
- Leitura de whitepapers e docs de cloud.

Ordem recomendada

- README avançado.
- API documentation.
- ADR.
- RFC.
- Technical proposal.

- Release notes.
- Incident report.
- Whitepapers e postmortems.

Conceitos obrigatórios

Context.

Problem.

Decision.

Alternatives.

Consequences.

Trade-offs.

Rollout.

Rollback.

Risks.

Monitoring.

Compatibility.

Práticas recomendadas

Escrever 1 ADR por semana.

Ler 1 postmortem por semana.

Documentar uma API usando inglês técnico.

Resumir uma decisão arquitetural.

Reescrever documentos antigos com mais clareza.

Exercícios técnicos

Escrever um ADR sobre uso de Kafka.

Escrever um RFC para criação de novo endpoint.

Criar documentação de API REST.

Escrever release notes de uma versão.

Criar um incident report simulado.

Ler um whitepaper e resumir os trade-offs.

Mini projeto

Criar documentação técnica completa de um microserviço de pedidos.

Documentos obrigatórios:

- README;
- API documentation;
- ADR de banco de dados;
- ADR de mensageria;
- RFC de nova feature;
- release notes;
- incident report simulado;
- arquitetura em texto.

Critérios de conclusão

O módulo está concluído quando o estudante consegue:

- escrever documentação compreensível por outro engenheiro;
- explicar decisões com trade-offs;
- documentar riscos e plano de rollback;
- ler documentação de cloud ou framework e aplicar;
- produzir material técnico assíncrono para time internacional.

Nível esperado ao finalizar

Intermediário forte, B1+ a B2 técnico.

Módulo 4 - Arquitetura, boas práticas e liderança técnica

Objetivo do módulo

Desenvolver comunicação em inglês para decisões arquiteturais, revisões técnicas, trade-offs e liderança de discussões.

Conteúdos a estudar

Architecture review.

System design vocabulary.

Trade-off discussion.

Technical leadership vocabulary.

Pushback professional.

Risk communication.

Scope negotiation.

Technical alignment.

Ordem recomendada

Vocabulário de arquitetura.

Explicação de trade-offs.

Revisão de arquitetura.

Comunicação de risco.

Discordância profissional.

Negociação de escopo.

Apresentação de decisão.

Simulação de reunião técnica.

Conceitos obrigatórios

Coupling.

Cohesion.

Scalability.

Maintainability.

Reliability.

Fault tolerance.

Latency.
Throughput.
Consistency.
Availability.
Observability.
Rollout strategy.
Rollback plan.
Operational complexity.

Práticas recomendadas

Apresentar 1 decisão arquitetural por semana.

Simular reunião de arquitetura.

Explicar uma escolha entre monólito modular e microsserviços.

Defender ou rejeitar uma abordagem.

Praticar frases de liderança técnica.

Exercícios técnicos

Explicar por que usar cache.

Explicar quando não usar microsserviços.

Defender uso de mensageria.

Rejeitar uma solução com acoplamento alto.

Propor rollout com feature flag.

Explicar impacto operacional de uma arquitetura.

Mini projeto

Apresentar oralmente o design de uma plataforma de e-commerce ou dashboard financeiro.

A apresentação deve cobrir:

- requisitos funcionais;
- requisitos não funcionais;
- APIs;
- banco de dados;
- cache;
- mensageria;
- segurança;
- observabilidade;
- deploy;
- riscos;
- trade-offs;
- evolução futura.

Critérios de conclusão

O módulo está concluído quando o estudante consegue:

- apresentar arquitetura em inglês;

- justificar decisões;
- apontar riscos;
- propor alternativas;
- discordar com maturidade;
- liderar uma discussão técnica de 20 a 30 minutos.

Nível esperado ao finalizar

B2 técnico com capacidade real de atuação em reuniões técnicas.

Módulo 5 - Produção, performance, segurança e escalabilidade

Objetivo do módulo

Preparar o estudante para falar, escrever e decidir em inglês sobre sistemas em produção.

Conteúdos a estudar

Incident calls.

Postmortems.

Observability.

Performance analysis.

Security considerations.

Scalability strategy.

Deployment strategy.

Rollback and mitigation.

Operational risks.

Cost awareness.

Ordem recomendada

Vocabulário de produção.

Observabilidade.

Incidentes.

Postmortems.

Performance.

Segurança.

Escalabilidade.

Custos e operação.

Conceitos obrigatórios

Error rate.

Latency.

Throughput.

Timeout.

Retry.

Circuit breaker.

Rate limiting.
Authentication.
Authorization.
Encryption.
Monitoring.
Alerting.
Logs.
Metrics.
Traces.
SLO.
SLA.
Rollback.
Root cause.
Mitigation.
Preventive action.

Práticas recomendadas

Simular incident call.
Escrever postmortem.

Explicar gráfico de latência.**Apresentar plano de rollback.**

Descrever risco de segurança.
Propor melhoria de observabilidade.

Exercícios técnicos**Explicar aumento de erro 500 em uma API.**

Descrever impacto de timeout em cadeia.
Escrever postmortem de falha em deploy.

Explicar como reduzir latência.**Defender uso de circuit breaker.****Explicar risco de expor dados sensíveis em logs.**

Propor plano de rollback.

Mini projeto

Criar um cenário de produção para uma API com falha simulada.

Entregáveis:

- incident report;
- timeline;
- root cause;
- mitigation;
- rollback plan;

- preventive actions;
- monitoring improvements;
- apresentação oral do incidente.

Critérios de conclusão

O módulo está concluído quando o estudante consegue:

- participar de incident call;
- escrever postmortem;
- explicar impacto em produção;
- discutir segurança;
- propor mitigação;
- avaliar performance;
- falar sobre escalabilidade e custos.

Nível esperado ao finalizar

B2+ técnico em contexto de produção.

Módulo 6 - Domínio sênior e entrevistas internacionais

Objetivo do módulo

Consolidar inglês técnico para entrevistas, liderança técnica e atuação sênior em times internacionais.

Conteúdos a estudar

Personal pitch.

Project deep dive.

Behavioral interview.

STAR method.

System design interview.

Live coding communication.

Architecture trade-offs.

Conflict resolution.

Incident response stories.

Salary negotiation.

Ordem recomendada

Pitch pessoal.

Projeto principal em inglês.

Respostas STAR.

Deep dive técnico.

System design.

Live coding em inglês.

Mock interviews.

Negociação.

Conceitos obrigatórios

Ownership.

Impact.

Scope.

Constraints.

Trade-offs.

Leadership.

Mentoring.

Conflict.

Delivery.

Reliability.

Business value.

Compensation.

Expectations.

Práticas recomendadas

Gravar pitch pessoal semanalmente.

Simular entrevista comportamental.

Resolver live coding narrando em inglês.

Apresentar 5 system designs.

Preparar histórias reais com método STAR.

Simular negociação salarial.

Exercícios técnicos

Criar pitch de 90 segundos.

Explicar principal projeto técnico em 5 minutos.

Criar 10 respostas STAR.

Fazer 5 apresentações de system design.

Fazer 20 entrevistas simuladas.

Explicar conflito técnico e resolução.

Negociar proposta em inglês.

Mini projeto

Preparar um pacote completo para entrevista internacional.

Entregáveis:

- personal pitch;
- resume bullets em inglês;
- LinkedIn summary técnico;
- project deep dive;
- 10 STAR answers;
- 5 system design scripts;
- salary negotiation script;

- lista de perguntas para entrevistador.

Critérios de conclusão

O módulo está concluído quando o estudante consegue:

- explicar trajetória profissional;
- defender projetos;
- responder perguntas comportamentais;
- desenhar sistemas;
- negociar tecnicamente;
- demonstrar liderança;
- participar de entrevista sem depender de tradução mental.

Nível esperado ao finalizar

B2+ funcional, com prontidão para entrevistas internacionais e colaboração técnica em inglês.

4. Critérios de domínio técnico por grande área

Reading técnico

Nível Domínio esperado

Iniciante Entende termos técnicos isolados e pequenos trechos de documentação.

Intermediário Lê documentação com apoio e aplica exemplos simples.

Avançado Lê documentação oficial, identifica limitações e aplica em projetos.

Sênior Usa documentação, RFCs e postmortems para tomar decisões técnicas e orientar o time.

Writing técnico

Nível Domínio esperado

Iniciante Escreve frases simples e commits básicos.

Intermediário Escreve PRs, bug reports e README com clareza razoável.

Avançado Produz ADRs, RFCs, documentação de API e relatórios técnicos.

Sênior Escreve documentos que orientam decisões arquiteturais, operação, evolução e governança técnica.

Speaking técnico

Nível Domínio esperado

Iniciante Fala frases curtas sobre tarefas simples.

Intermediário Participa de daily, explica bugs e features conhecidas.

Avançado Participa de reuniões técnicas, code reviews e discussões de arquitetura.

Sênior Lidera discussões, defende trade-offs, negocia escopo e conduz decisões técnicas.

Listening técnico

Nível Domínio esperado

Iniciante Entende vídeos lentos e frases previsíveis.

Intermediário Entende reuniões simples com apoio de contexto.

Avançado Entende reuniões técnicas, entrevistas e talks com sotaques variados.

Sênior Entende discussões complexas em tempo real e responde no fluxo.

Entrevistas internacionais

Nível Domínio esperado

Iniciante Consegue se apresentar de forma simples.

Intermediário Explica experiência, stack e projetos básicos.

Avançado Responde behavioral, live coding e project deep dive.

Sênior Defende system design, trade-offs, liderança, incidentes e negociação salarial.

5. Aplicação prática em projetos reais

APIs

O inglês técnico aparece em:

- documentação de endpoints;
- descrição de contratos;
- release notes;
- bug reports;
- breaking changes;
- autenticação e autorização;
- versionamento.

Exemplo:

This endpoint returns the current status of an order based on its unique identifier.

Visão sênior:

- documentar compatibilidade;
- explicar impacto de mudança no contrato;
- justificar versionamento;
- avaliar segurança;
- planejar rollout.

Banco de dados

Aplicações:

- documentação de schema;
- decisão entre SQL e NoSQL;
- explicação de índices;
- migrações;
- consistência;
- transações;
- performance.

Exemplo:

We added an index to improve query performance for customer lookups by document number.

Visão sênior:

- explicar trade-off entre normalização e performance;
- avaliar impacto de migração;
- planejar rollback;

- considerar locks, carga e consistência.

Microserviços

Aplicações:

- service boundaries;
- comunicação síncrona e assíncrona;
- ownership;
- contratos;
- observabilidade;
- resiliência.

Exemplo:

We decided to split the billing logic into a separate service because it has a different lifecycle and scaling profile.

Visão sênior:

- evitar microserviços prematuros;
- justificar boundaries;
- avaliar complexidade operacional;
- definir estratégia de integração.

Mensageria

Aplicações:

- Kafka;
- RabbitMQ;
- eventos;
- retry;
- dead-letter queue;
- idempotência;
- consistência eventual.

Exemplo:

The event-driven approach decouples order creation from payment processing.

Visão sênior:

- explicar delivery semantics;
- garantir idempotência;
- lidar com duplicidade;
- observar lag;
- planejar reprocessamento.

Segurança

Aplicações:

- autenticação;
- autorização;
- OAuth2;
- JWT;
- criptografia;

- dados sensíveis;
- logs;
- rate limiting.

Exemplo:

We should avoid logging sensitive customer data because it increases compliance and security risks.

Visão sênior:

- avaliar threat model;
- revisar exposição de dados;
- definir controles;
- alinhar segurança com arquitetura.

Observabilidade**Aplicações:**

- logs;
- métricas;
- traces;
- dashboards;
- alertas;
- SLO/SLA;
- incident response.

Exemplo:

We need better tracing to understand where the latency is being introduced.

Visão sênior:

- definir indicadores relevantes;
- evitar alert fatigue;
- priorizar observabilidade antes de escalar;
- usar dados para decisões.

Testes**Aplicações:**

- unit tests;
- integration tests;
- contract tests;
- regression tests;
- test coverage;
- flaky tests.

Exemplo:

This test covers the null payload scenario and prevents regression in the mapper logic.

Visão sênior:

- decidir estratégia de testes;
- equilibrar custo e cobertura;
- revisar qualidade dos testes;

- evitar testes frágeis.

Deploy

Aplicações:

- CI/CD;
- rollback;
- feature flags;
- canary release;
- blue-green deployment;
- release notes.

Exemplo:

The rollout plan is to enable the feature for 10% of users and monitor error rates before full release.

Visão sênior:

- reduzir risco operacional;
- planejar rollback;
- alinhar deploy com observabilidade;
- documentar impacto.

Arquitetura

Aplicações:

- ADR;
- RFC;
- system design;
- trade-offs;
- escalabilidade;
- manutenção;
- evolução.

Exemplo:

The main trade-off is that the asynchronous flow improves resilience but makes debugging more complex.

Visão sênior:

- justificar decisões;
- antecipar riscos;
- avaliar custo técnico;
- projetar evolução gradual.

Performance

Aplicações:

- latência;
- throughput;
- cache;
- queries;
- profiling;

- gargalos;
- escalabilidade.

Exemplo:

The current bottleneck is the database query, not the application layer.

Visão sênior:

- diagnosticar com dados;
- evitar otimização prematura;
- avaliar custo da solução;
- medir antes e depois.

Manutenção evolutiva**Aplicações:**

- refactoring;
- technical debt;
- backward compatibility;
- migration strategy;
- documentation;
- deprecation.

Exemplo:

This refactoring reduces duplication without changing the public API contract.

Visão sênior:

- evoluir sem quebrar produção;
- comunicar riscos;
- documentar migrações;
- reduzir dívida técnica com estratégia.

6. Profundidade sênior por tópico

Trade-offs

Um sênior precisa explicar que toda decisão tem custo.

Exemplo:

Using Kafka improves decoupling and resilience, but it increases operational complexity and requires stronger observability.

Decisões arquiteturais

Deve conseguir justificar escolhas com base em:

- requisitos funcionais;
- requisitos não funcionais;
- time disponível;
- maturidade do time;
- custo de operação;
- risco;
- prazo;

- evolução futura.

Impacto em produção

Deve sempre considerar:

- erro;
- latência;
- carga;
- rollback;
- segurança;
- monitoramento;
- compatibilidade;
- dados reais.

Escalabilidade

Deve saber explicar:

- escala horizontal;
- gargalos;
- filas;
- cache;
- particionamento;
- limites de banco;
- limites de infraestrutura.

Manutenibilidade

Deve saber defender:

- código simples;
- baixo acoplamento;
- alta coesão;
- documentação;
- testes;
- padrões consistentes;
- boundaries claros.

Segurança

Deve comunicar riscos como:

- vazamento de dados;
- permissões excessivas;
- autenticação fraca;
- ausência de validação;
- exposição de secrets;
- logs sensíveis.

Performance

Deve evitar afirmações vagas e usar evidências:

Based on the metrics, the average latency increased from 120ms to 450ms after the last deployment.

Custos técnicos

Deve saber explicar dívida técnica:

This shortcut may speed up delivery now, but it will make future changes more expensive.

Antipadrões

Evitar:

- documentação inexistente;
- decisões sem ADR;
- PRs sem contexto;
- arquitetura sem rollback;
- microsserviços sem observabilidade;
- incidentes sem postmortem;
- comunicação agressiva;
- aceitar escopo inviável sem questionar.

Estratégias de evolução

Um sênior propõe evolução gradual:

- feature flags;
- rollout progressivo;
- refactoring incremental;
- compatibilidade retroativa;
- métricas antes/depois;
- plano de rollback;
- documentação da decisão.

7. Plano de execução sugerido

Duração recomendada

Plano base: 24 semanas.

Carga ideal:

- mínimo: 30 minutos por dia;

recomendado: 45 a 60 minutos por dia;

intensivo: 90 minutos por dia.

Para quem já está em B1, 24 semanas podem levar a um B2 técnico funcional. Para quem está em A2, pode ser necessário repetir ciclos e estender para 36 a 48 semanas.

Distribuição semanal recomendada

Segunda-feira - Reading técnico

30 a 60 minutos de documentação oficial.

Anotar vocabulário.

Resumir em inglês o que foi entendido.

Terça-feira - Writing técnico

Escrever commits, PRs, bug reports ou ADRs.

Revisar clareza, estrutura e objetividade.

Quarta-feira - Speaking técnico

Gravar explicação de bug, feature ou arquitetura.

Repetir até ficar natural.

Quinta-feira - Listening técnico

Ouvir podcast, talk ou reunião técnica.

Anotar decisões, problemas e expressões.

Sexta-feira - Entrevista ou liderança técnica

Simular pergunta comportamental.

Praticar system design.

Treinar frases de pushback, risco e trade-off.

Sábado - Projeto de fixação

Criar documentação, apresentação ou simulação prática.

Domingo - Revisão

Revisar vocabulário.

Regravar pitch.

Ajustar erros.

Fazer checkpoint semanal.

Cronograma por semanas

Semanas Foco Resultado esperado

1-4 Fundamentos técnicos Ler docs simples, escrever commits e explicar tarefas básicas.

5-8 Rotina de desenvolvimento

Participar de daily, PR, bug report e code review em inglês.

9-12 Documentação avançada Escrever README, ADR, RFC, API docs e release notes.

13-16 Arquitetura e liderança Explicar trade-offs, riscos, rollout e decisões arquiteturais.

17-20 Produção e incidentes Conduzir incident report, postmortem e discussões de observabilidade.

21-24 Entrevistas internacionais Fazer pitch, STAR answers, system design e mock interviews.

Checkpoints técnicos**Checkpoint 1 - Semana 4****Entregáveis:**

- 30 commits em inglês;
- 5 pequenos resumos de documentação;
- 3 áudios explicando tarefas;
- 1 README simples.

Checkpoint 2 - Semana 8**Entregáveis:**

- 5 PR descriptions;
- 5 bug reports;

- 5 simulações de daily;
- 3 code review discussions.

Checkpoint 3 - Semana 12

Entregáveis:

- 3 ADRs;
- 1 RFC;
- 1 documentação de API;
- 1 release note;
- 1 technical proposal.

Checkpoint 4 - Semana 16

Entregáveis:

- 3 apresentações de arquitetura;
- 5 explicações de trade-offs;
- 3 simulações de pushback técnico;
- 1 plano de rollout.

Checkpoint 5 - Semana 20

Entregáveis:

- 2 incident reports;
- 2 postmortems;
- 2 simulações de incident call;
- 1 apresentação sobre observabilidade.

Checkpoint 6 - Semana 24

Entregáveis:

- pitch pessoal;
- projeto principal explicado em inglês;
- 10 respostas STAR;
- 5 system designs;
- 20 mock interviews;
- script de negociação salarial.

8. Projetos recomendados para portfólio

Projeto 1 - API documentada em inglês

Criar uma API Spring Boot ou Node.js com documentação completa.

Entregáveis:

- README;
- endpoints;
- examples;

- testing instructions;
- deployment instructions;
- release notes.
- Competência treinada: writing técnico.

Projeto 2 - ADR collection

Criar uma pasta /docs/adr com decisões arquiteturais.

Exemplos:

escolha de banco relacional;

- uso de Kafka;
- uso de Redis;
- estratégia de autenticação;
- padrão arquitetural;
- estratégia de deploy.
- Competência treinada: arquitetura em inglês.

Projeto 3 - Incident simulation

Simular um incidente em produção.

Entregáveis:

- incident report;
- timeline;
- root cause;
- mitigation;
- rollback plan;
- preventive actions.
- Competência treinada: produção, incident response e postmortem.

Projeto 4 - System design presentation

Preparar apresentação oral de um sistema.

Sugestões:

- order management system;
- financial dashboard;
- authentication service;
- file ingestion platform;
- e-commerce checkout;
- notification system.
- Competência treinada: speaking técnico e entrevistas.

Projeto 5 - International interview package

Criar kit completo para entrevistas.

Entregáveis:

- personal pitch;

- resume bullets;
- project deep dive;
- STAR answers;
- system design scripts;
- salary negotiation script.
- Competência treinada: empregabilidade internacional.

9. Checklist de domínio técnico

Reading

- Leio documentação oficial sem depender de tradução.
- Entendo warnings, limitations e caveats.
- Consigo aplicar exemplos em projeto real.
- Consigo resumir documentação para outro dev.
- Leio issues, PRs e RFCs com autonomia.

Writing

- Escrevo commits claros.
- Escrevo PRs com contexto, impacto e testes.
- Escrevo bug reports reproduzíveis.
- Escrevo ADRs com trade-offs.
- Escrevo incident reports objetivos.
- Documento APIs e arquitetura com clareza.

Speaking

- Consigo participar de daily.
- Explico bugs e features.
- Defendo estimativas.
- Explico decisões arquiteturais.
- Discordo de forma profissional.
- Apresento projeto técnico em inglês.

Listening

- Entendo vídeos técnicos sem legenda.
- Entendo reuniões com sotaques variados.
- Identifico decisões e próximos passos.
- Respondo perguntas no fluxo.
- Consigo acompanhar incident calls.

Entrevistas

- Tenho pitch pessoal.
- Explico meu principal projeto.
- Tenho 10 respostas STAR.
- Apresentei 5 system designs.
- Fiz 20 mock interviews.
- Sei negociar salário em inglês.

10. Indicadores de prontidão para nível avançado/sênior

O estudante está pronto para atuar em nível avançado/sênior em inglês quando consegue:

- Ler documentação oficial e aplicar sem tradução.
- Escrever PRs e ADRs claros para times internacionais.

Explicar bugs, features e decisões arquiteturais oralmente.

Participar de reuniões técnicas sem travar.

Entender sotaques variados em contexto técnico.

Defender trade-offs com clareza.

Recusar escopo inviável com argumentos profissionais.

Conduzir ou participar de postmortems.

Apresentar system design em inglês.

Passar por entrevistas internacionais com boa comunicação.

Negociar salário e expectativas de forma objetiva.

Comunicar risco, impacto, rollout e rollback.

Produzir documentação que ajuda o time a tomar decisões.

Demonstrar maturidade técnica e comunicação de liderança.

Resumo final da trilha

Esta trilha transforma o inglês em uma ferramenta profissional para engenharia de software. O foco não é gramática isolada, mas sim uso real do idioma em ambientes técnicos.

Ao final, o estudante deve ser capaz de:

- ler documentação técnica complexa;
- escrever documentos profissionais;
- participar de reuniões internacionais;
- explicar arquitetura;
- conduzir discussões técnicas;
- lidar com incidentes;
- fazer entrevistas internacionais;
- defender decisões como profissional avançado/sênior.
- O resultado esperado é um profissional com B2+ funcional, capaz de trabalhar em times globais com autonomia, clareza e maturidade técnica.